Weak-Memory Local Reasoning, Revisited

Ian Wehrman

Draft of November 13, 2026

Abstract

My 2012 dissertation proposed a program logic for local reasoning about concurrent programs on an x86-like, total-store-order memory model. It left two components unfinished: the soundness proof, and a proof system for assertions. Out of curiosity, I have returned to both with a mechanization in Rocq. The soundness theorem turns out not to hold as stated, because four of the original rules derive false specifications. However, a repaired calculus is sound and adequate for the machine, and it keeps most of the original design: the assertion language, the unit law for **emp**, the temporal frame rule, and the original load and store axioms. This report recalls the original model and rules for a reader who knew the dissertation but has forgotten its details. It then explains what was wrong, what had to change and why, and what the repaired logic can prove.

1 Introduction

The dissertation [12] set out to give a local reasoning principle for programs running on an x86-like machine, where each processor buffers its writes and other processors cannot see a write until it leaves the buffer. Its logic has three separating conjunctions. Spatial conjunction $P * Q$ splits memory by address, as in separation logic. Temporal conjunction $P \lhd Q$ splits it by time: P describes earlier writes and Q later ones, possibly to the same addresses. Strong temporal conjunction $P \blacktriangleleft Q$ requires both. Each has a frame rule, and a concurrency rule and a shared invariant in the style of concurrent separation logic sit on top.

The dissertation was candid about where it stopped. Its statement of contributions lists two major components as unfinished and admits a third gap:

1. there is no proof system for the assertion language;
2. the soundness proof for the specification logic is incomplete; and
3. there are few worked examples, so it is unclear how one would actually use the rules to verify a program.

It also hypothesized that the logic was sound and that the remaining proofs were "mostly straightforward inductions."

I have wondered since whether that was true. This report returns to those three items, fourteen years later, to satisfy that curiosity, with a mechanization in the Rocq prover. Its findings are as follows.

The soundness theorem does not hold as stated. Several auxiliary claims are false, but each can be fixed without changing the design (Section 3.1). More seriously, the rules themselves are inconsistent with any adequate semantics: from SKIP, CONS, INV and FRAME-TM alone one can derive that a skip changes the contents of a memory cell (Section 3.2). No choice of safety predicate can repair that.

The cause is general. An unrestricted temporal frame rule, a right unit for temporal conjunction, the rule of consequence, and a command that gives away ownership of a cell cannot coexist once some history that held before the transfer is false after it (Section 3.3). The observation is independent of the memory model, and a sequential conjunction in which later values override earlier ones supplies such histories.

A repaired calculus is sound. The repair replaces the invariant rule by lock and unlock rules that transfer the invariant, restricts the temporal frame rule to commands that do not release the lock, and adjusts four side conditions (Section 4). The repaired calculus is proved sound by induction on derivations, with an adequacy theorem for the dissertation's machine (Section 7). The proofs are checked by the Rocq kernel and use no axioms.

Most of the design survives. The machine, the assertion language and its interpretation of atoms, the unit laws for **emp**, overlapping temporal history, the temporal and spatial frame rules, the concurrency rule, and the original load and store axioms are all retained. So is the dissertation's assertion algebra, with one family of exceptions: the original laws that rebracket or exchange through strong temporal conjunction fail, and with them the strong temporal frame rule. The exchange laws between spatial and temporal conjunction also fail or are vacuous (Appendix D).

Assertions have a proof system. A calculus of 47 rules for entailment between assertions is proved sound (Section 6.1). Its completeness is not established and was not a goal, but it is enough: the larger derivations below use no semantic entailment at all.

The logic can be used. Section 8 derives a shared container in which a thread acquires the lock, removes a cell, releases the lock, writes the cell privately, and later returns it, while another processor's write stays buffered throughout. It re-derives the dissertation's own message-passing example. And it builds named locks on top of the single global lock, as derived rules, and uses one to pass a message of any length from a producer to a consumer.

This report is for a reader who knew the original dissertation but has forgotten its details. Section 2 recalls the original model and calculus in the original notation. Sections 3 and 4 say what was wrong and what changed. Sections 5 to 8 present the repaired model, calculus, results and derivations. Proofs are not reproduced; every claim cites the name of its Rocq theorem, and the appendix gives fuller statements.

2 The original model and calculus

This section recalls Chapter 4 of the dissertation, in its notation. Nothing here is new. Page and figure numbers refer to the dissertation.

2.1 The machine

A *memory system* is a triple $\mu = (h, B, K)$: a heap $h : \mathbb{L} \rightharpoonup \mathbb{V}$, an array of write buffers $B : \mathbb{P} \to (\mathbb{L} \times \mathbb{V})$ list with the oldest write first, and a set $K \subseteq \mathbb{P}$ of *blocked* processors. A state pairs a stack s with a memory system. The lock is not a separate component: it is free when $K = \emptyset$ and held by i when $K = \mathbb{P} \setminus \{i\}$.

Primitive commands carry the identifier of the processor that executes them. Their behaviour on processor i is as follows.

- $[e] := e'_i$ appends a write to $B(i)$. The address must be allocated, in the heap or in $B(i)$. A blocked processor may still store.
- $x := [e]_i$ requires $i \notin K$ and returns the newest write to that address in $B(i)$, or else the heap value.
- fence_i, lock_i and unlock_i require $B(i) = \varepsilon$. lock_i additionally requires $K = \emptyset$ and sets $K = \mathbb{P} \setminus \{i\}$; unlock_i reverses this.
- A silent step $\underset{\tau}{\rightarrow}$ moves the oldest write of some unblocked processor from its buffer to the heap.

Commands are built from primitives by sequencing $c\,;c'$, choice $c+c'$, iteration c^* and parallel composition $c \parallel c'$. There is no allocation. While a processor holds the lock, no other processor can load or flush, but others may continue to store and to assign to registers.

2.2 Generalized memories and separation

A *generalized memory system* $\nu = (h, B, K, \Gamma)$ adds a *buffer-completeness* set $\Gamma \subseteq \mathbb{P}$, with $h = \emptyset$ whenever $\Gamma = \emptyset$. Membership $i \in \Gamma$ means that buffer i is complete: nothing may be placed before it.

Three operations combine generalized memories. Write $\nu_k = (h_k, B_k, K_k, \Gamma_k)$. All three take $K_0 \cup K_1$ and $\Gamma_0 \cup \Gamma_1$ and require $K_0 \cap K_1 = \emptyset$.

Spatial, $\nu_0 \mathbin{\widehat{*}} \nu_1$. The address domains of the two sides, counting both heaps and buffers, are disjoint. The heap is $h_0 \uplus h_1$ and each buffer is an interleaving of the two.

Temporal, $\nu_0 \mathbin{\widehat{\lhd}} \nu_1$. The heap is $h_0 \backslash\!\backslash h_1$, so later values win, and each buffer is $B_0(i) +\!\!+ B_1(i)$. The sides may share addresses, subject to a compatibility condition: an unblocked pending write in ν_0 may not target an address in the heap of ν_1, nor an address pending on a different unblocked processor in ν_1. Finally $B_0(i) = \varepsilon$ for every $i \in \Gamma_1$.

Spatiotemporal, $\nu_0 \mathbin{\widehat{\#}} \nu_1$. As temporal, but each buffer is any interleaving whose lookup function agrees with $B_0(i) +\!\!+ B_1(i)$, and the condition on Γ_1 is dropped. It contains both of the others and is used only in the semantics of specifications.

Strong temporal separation $\nu_0 \mathbin{\widehat{\blacktriangleleft}} \nu_1$ is $\nu_0 \mathbin{\widehat{\lhd}} \nu_1$ restricted to pairs that are also spatially compatible.

2.3 Assertions

Assertions are given by

$$\begin{aligned} P \;::=\; & b \mid P \vee P \mid P \wedge P \mid \exists x : P \mid \forall x : P \mid \mathbf{emp} \mid \mathbf{bar}_e \mid \mathbf{lock}_e \mid e \rightsquigarrow_{e'} e'' \\ & \mid P * P \mid P \lhd P \mid P \mathrel{-\!\!*} P \mid P \mathrel{-\!\!\lhd} P \mid P \mathrel{\lhd\!\!-} P \end{aligned}$$

and are interpreted over a stack and a generalized memory. The atoms are:

- **emp**: empty heap, empty buffers, $K = \emptyset$; Γ is unconstrained.
- $\mathbf{bar}_i$: as **emp**, with $i \in \Gamma$ when i is a processor.
- $\mathbf{lock}_i$: empty heap and buffers, with $K = \mathbb{P} \setminus \{i\}$.
- $e \rightsquigarrow_i e'$ ("leads to"): either a single *pending* write (e, e') in $B(i)$ with an empty heap and $K \subseteq \{i\}$, or a single *flushed* heap cell $e \mapsto e'$ with empty buffers, $K = \emptyset$ and $i \in \Gamma$.

$P * Q$ and $P \lhd Q$ hold of ν when ν is a spatial, respectively temporal, combination of a model of P and a model of Q. The three implications are the corresponding adjoints. Two abbreviations matter below:

$$P \blacktriangleleft Q \;=_{df}\; (P * Q) \wedge (P \lhd Q) \qquad\qquad e \mapsto e' \;=_{df}\; (\exists x : e \rightsquigarrow_x e') \lhd (\forall x : \mathbf{bar}_x).$$

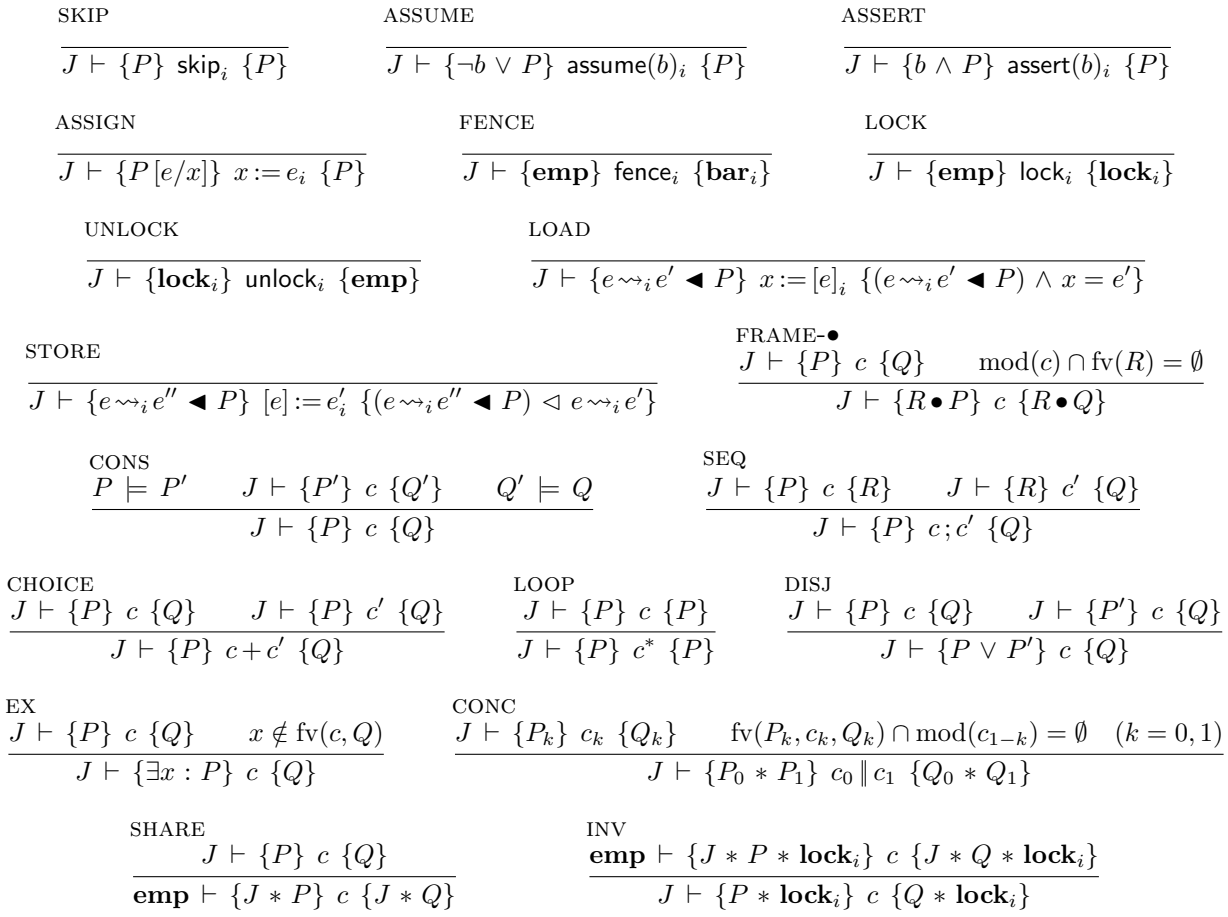


Figure 1: The original axioms and rules (Figures 4.7 and 4.8). FRAME-• stands for three rules, FRAME-SP, FRAME-TM and FRAME-STM, with • read as $*$, $\lhd$ and $\blacktriangleleft$.

Points-to therefore describes a flushed cell with *every* processor buffer-complete. The dissertation claims that every assertion is closed under flushing (Proposition 9), and that each conjunction is associative with **emp** as a unit (Figure 4.5).

2.4 Specifications and rules

A specification $J \vdash \{P\}\ c\ \{Q\}$ has an invariant J for shared state and a private precondition and postcondition. Figure 1 reproduces the axioms and rules.

Three features of the design are worth recalling, because the repair turns on them. First, LOCK and UNLOCK produce and consume only the token $\mathbf{lock}_i$. The invariant reaches the lock holder through INV, which may be applied to any command executed while the lock is held and allows the private state to exchange cells with J. Second, FRAME-TM has only the usual variable condition: any history R may be placed before any specification. Third, temporal conjunction is not cancellative, as the dissertation remarks: $\mu \mathbin{\widetilde{\lhd}} \mu_0 = \mu \mathbin{\widetilde{\lhd}} \mu_1$ does not imply $\mu_0 = \mu_1$.

2.5 Semantics and status

Truth of a specification is defined through a family of predicates $safe_n(c, s, \mu, J, Q)$ after Vafeiadis. The predicate holds at $n+1$ when: if c is skip then $(s, \mu, \mathbb{P}) \models Q$; and for every μ_J, μ_F and $\mu_0 \in \mu_J \mathbin{\widetilde{*}} (\mu_F \mathbin{\widetilde{\#}} \mu)$ that is lock-complete, with either $(s, \mu_J, \mathbb{P}) \models J$ or the lock held in μ_0, the configuration does not abort, and every step to (c', s', μ_1) admits a new decomposition $\mu_1 \in \mu'_J \mathbin{\widetilde{*}} (\mu'_F \mathbin{\widetilde{\#}} \mu')$ with $\mu'_F \preceq \mu_F$, the same condition on J, and $safe_n(c', s', \mu', J, Q)$. A specification is true, $J \models \{P\}\ c\ \{Q\}$, when $(s, \mu, \mathbb{P}) \models P$ implies $safe_n(c, s, \mu, J, Q)$ for all n. Local states are always read at full completeness, $\Gamma = \mathbb{P}$.

Theorem 1 states that provable specifications are true. The dissertation gives soundness lemmas for the axioms in an appendix, sketches a frame lemma for the spatiotemporal operator $\mathbin{\widetilde{\#}}$, and reduces the spatial frame rule to Conjecture 1, which is left open. The conjecture says that spatial separation survives a step: if a memory splits spatially into μ_b and μ_c and takes a step, then any spatiotemporal split of the result into a flushed descendant of μ_b and some μ'_c is again a spatial split. Associativity of buffer overriding (Proposition 2) and of the three separations, and flushing closure of assertions (Proposition 9), are stated with at most a sketch.

3 What was wrong

The problems are presented from the smallest to the most serious, which is also the order in which the mechanization found them. Each is a theorem checked by Rocq, and the theorem's name is given so that the statement can be inspected. All of them are stated against a transcription of the original definitions.

3.1 Errors that can be fixed in place

Table 1 lists six original claims that are false as stated. None is deep: each is fixed by correcting the one definition or side condition involved, and the rest of the design is unaffected.

Two remarks. The repaired compatibility relation is stronger than the original one, so it rejects some compositions the dissertation admits. Those are states in which a blocked processor has a pending write to a cell that someone else owns. And the repaired implications are the ordinary Kripke reading of implication over the flushing preorder; the three adjunctions between conjunctions and implications still hold.

The failure of UNLOCK points at something larger. The original safety predicate asks for the invariant only while the lock is free. Once a thread holds the lock, the predicate has forgotten that J held at the moment of acquisition, so neither UNLOCK nor INV can be justified from it; INV on its own takes a true specification to a false one (`original_inv_alone_counterexample`). That alone would call for a different predicate. The next subsection shows that it calls for different rules.

3.2 The original rules are inconsistent

The derivations below use the dissertation's own points-to, $x \mapsto v$, and are checked against its own interpretation of assertions.

History can be invented. A model is a memory and records nothing of how that memory came about. So a current cell is also a model of any stale value followed by that cell:

$$x \mapsto 7 * \mathbf{lock}_i \ \models\ x \mapsto 5 \lhd (x \mapsto 7 * \mathbf{lock}_i). \tag{1}$$

Original claim	Counterexample	Repair
ASSIGN is sound for every invariant	With J the assertion $x = 0$, the axiom derives $J \vdash \{\mathbf{true}\}\ x := 1_1\ \{\mathbf{true}\}$, and the assignment falsifies J with the lock free. `original_assignment_invalid`	Require $x \notin \mathrm{fv}(J)$.
Buffer overriding is associative (Prop. 2)	Three one-element buffers writing 1, 2, 3 to one address: the list $[2,1,3]$ is in $a \backslash\backslash (b \backslash\backslash c)$ but not in $(a \backslash\backslash b) \backslash\backslash c$. `original_buffer_override_not_associative`	A write from the later buffer may be taken only after the earlier buffer's writes to that address.
Spatiotemporal separation is associative	Compatibility ignores blocked processors, so whether two memories compose can depend on a third that supplies the blocking fact. `original_spatiotemporal_not_associative`	Check conflicts whether or not the processor is blocked.
Assertions are closed under flushing (Prop. 9)	$\mathbf{false} \lhd\!\!- \mathbf{bar}_1$ holds vacuously while processor 1 has a pending write, and fails once the write flushes. `original_flushing_closure_false`	Implications quantify over all futures of the current memory.
Conjecture 1: after a step, a spatiotemporal split of a spatially split memory is spatial, if its first part is a flushed descendant of the original first part	A cell duplicated across $\widetilde{\#}$ with equal values is not a spatial split. `original_conjecture1_false`	Not needed by the final proof.
UNLOCK is sound for every invariant	With $J = \mathbf{false}$ and the lock held, unlock_i is enabled and nothing restores J. `original_arbitrary_invariant_unlock_invalid`	See Section 4.

Table 1: Original claims that fail, each with a checked counterexample. Appendix C states the three repaired definitions.

The stale cell is overridden by the later one, and nothing remains of it. This is the non-cancellativity that the dissertation notes in passing.

A skip that changes memory. Let J be $\mathbf{emp} \vee x \mapsto 7$. The following is a derivation in the original calculus.

1. $\mathbf{emp} \vdash \{J * x \mapsto 7 * \mathbf{lock}_i\}\ \mathsf{skip}_i\ \{J * \mathbf{emp} * \mathbf{lock}_i\}$, by SKIP and CONS: J cannot be its second disjunct, since the cell would occur twice, and in the first disjunct the cell can be reassigned to J.
2. $J \vdash \{x \mapsto 7 * \mathbf{lock}_i\}\ \mathsf{skip}_i\ \{\mathbf{emp} * \mathbf{lock}_i\}$, by INV. The thread has given its cell to the invariant.
3. $J \vdash \{x \mapsto 5 \lhd (x \mapsto 7 * \mathbf{lock}_i)\}\ \mathsf{skip}_i\ \{x \mapsto 5 \lhd (\mathbf{emp} * \mathbf{lock}_i)\}$, by FRAME-TM.
4. $J \vdash \{x \mapsto 7 * \mathbf{lock}_i\}\ \mathsf{skip}_i\ \{x \mapsto 5 * \mathbf{lock}_i\}$, by CONS, using (1) on the left and the unit law for **emp** on the right.

The conclusion says that a thread holding the lock and a cell containing 7 may do nothing and then own that cell containing 5. In the state where processor 1 holds the lock and x contains 7, the precondition holds and the postcondition does not. (`original_points_stale_value_unsound`; the mechanized statement parses INV's premise as $J * (P * \mathbf{lock}_i)$.)

A skip that drains another processor's buffer. Spatial conjunction takes the union of the completeness sets and does not require them to be disjoint, so an empty neighbour can copy a

cell's marks: $x \mapsto 7 * \mathbf{emp} \models x \mapsto 7 * \mathbf{bar}_2$. With J the cell $x \mapsto 7$, SKIP, CONS and INV give $J \vdash \{\mathbf{emp} * \mathbf{lock}_1\}\ \mathsf{skip}_1\ \{\mathbf{bar}_2 * \mathbf{lock}_1\}$, and FRAME-TM with a pending write $R = y \rightsquigarrow_2 17$ gives

$$J \vdash \{R \lhd (\mathbf{emp} * \mathbf{lock}_1)\}\ \mathsf{skip}_1\ \{R \lhd (\mathbf{bar}_2 * \mathbf{lock}_1)\}\,.$$

When processor 1 holds the lock and processor 2's write to y is still buffered, the precondition holds. The postcondition places a barrier for processor 2 after its pending write, which that memory does not satisfy. (`original_points_foreign_barrier_unsound`.)

The strong temporal frame rule does not escape. One might hope that FRAME-STM is safe, since its frame is also spatially separate. It is not, because $R \blacktriangleleft P$ abbreviates $(R * P) \wedge (R \lhd P)$ and the two conjuncts may be witnessed by different splits. Take $R = \mathbf{emp} \vee x \mapsto 5$. In the state above, the spatial conjunct of $R \blacktriangleleft (x \mapsto 7 * \mathbf{lock}_1)$ holds with R empty, and the temporal conjunct holds with R the stale cell. Applying FRAME-STM to step 2 yields a postcondition that the unchanged state cannot satisfy. (`original_strong_frame_counterexample`; this derivation uses INV and FRAME-STM but not FRAME-TM.)

All three conclusions are also invalid for the original safety predicate, but that is the lesser point. They are false under the dissertation's informal reading of a specification, and under any semantics for which a true specification describes what the program does.

3.3 Why no semantics could repair it

The first derivation is an instance of a general conflict.

Proposition 1 (`trilemma`)**.** *Suppose a logic of triples has an unrestricted temporal frame rule, the rule of consequence, and the entailment $R \lhd \mathbf{emp} \models R$. Suppose $\{P\}\ c\ \{\mathbf{emp}\}$ is provable. Then $\{R \lhd P\}\ c\ \{R\}$ is provable for every R. Hence if some execution of c starts in a state satisfying $R \lhd P$ and ends in a state that does not satisfy R, the logic is unsound.*

The proof is two rule applications. The content is in the last sentence. A triple $\{P\}\ c\ \{\mathbf{emp}\}$ with P owning a cell is a *transfer-out contract*: after c, the cell belongs to someone else. Whenever temporal conjunction lets later values override earlier ones, R may describe an older state of the same cell, as the store axiom's own postcondition does. Framing R across the transfer then hands the thread a stale copy of a cell it no longer owns.

So four things cannot all be kept: the unrestricted temporal frame rule, the right-unit law, transfer-out contracts ending in **emp**, and overlapping temporal history. A sound logic must give one up, and the choice determines its character:

- *No transfer out.* Cells may be taken from the invariant and never returned. This is sound and useless for a shared container.
- *No right unit.* Let **emp** record that a cell was given away. Framing survives, and every postcondition that mentions **emp** changes meaning.
- *Transfer leaves a mark.* Return an explicit tombstone assertion instead of **emp**. The algebra survives; every later specification carries the tombstones.
- *Restrict the frame.* Forbid temporal frames that overlap the cells being given away. Everything else is kept.

The first two were built and abandoned during this work, for the reasons given. The repaired calculus takes the fourth. Proposition 1 shows that some restriction of this kind is needed, and the derivations above supply histories that do become false after a transfer. It does not show that the particular restriction adopted is the weakest possible.

The second derivation has a different cause and a different repair. The all-processor completeness of points-to, and more generally the completeness marks of whatever the invariant holds, should not pass to a thread that acquires the lock. A thread can vouch only for its own buffer.

3.4 The dissertation's own example

Section 4.1 of the dissertation proves a small message-passing program correct, using an invariant that mentions two registers and the derived rule ATOMIC. That proof does not carry over to the repaired calculus: ATOMIC and SHARE in their original forms, and auxiliary-variable elimination, are not available, and the concurrency rule now forbids either thread from modifying a variable free in the invariant. The proof does not carry over, although its conclusion is still true. Section 8.4 derives the same program from a different invariant and recovers the original conclusion.

4 What changed

Table 2 lists every difference between the original calculus and the one proved sound. Each is marked as an error (the original statement is false, with a checked counterexample), as forced by an error elsewhere, or as a choice among sound alternatives.

Original	Repaired	Kind
ASSIGN, LOAD, EX, CONC say nothing of $\mathrm{fv}(J)$	The assigned, loaded or bound variable, and every variable a parallel branch modifies, is absent from $\mathrm{fv}(J)$	error
List overriding by lookup; compatibility ignores blocked processors; pointwise implications	Ordered overriding; stable compatibility; implications quantify over futures (Table 1)	error
LOCK and UNLOCK exchange a token; INV moves cells	INV is removed; LOCK hands over J and UNLOCK takes it back	error
FRAME-TM for every specification	FRAME-TM only for commands that contain no unlock	error
FRAME-STM	Omitted. Sound for one common split of the frame, unsound for $(R * P) \wedge (R \lhd P)$	error
SHARE	Holds as a theorem outside the calculus, and only when the precondition and postcondition are lock-free	forced
$e \mapsto e'$ with every processor complete	$e \rightarrowtail e'$, complete for some processor; load and store rules for it	forced
Any assertion as invariant	Invariants from a guarded, buffer-free fragment	choice
Any set of processors	At least two active processors	forced

Table 2: Differences between the original and the repaired calculus.

Three of these need a word of explanation.

Transferring lock rules. Section 3.1 observed that the original predicate forgets the invariant at acquisition. The standard cure, from concurrent separation logic, is to move ownership with the lock: acquiring gives the thread the cells of J, and releasing requires it to give back cells satisfying J. INV then has nothing left to do. Removing it is also what blocks the three derivations of Section 3.2, all of which are proofs about skip: restricting the frame rules alone would not have been enough.

The cell. A thread that acquires the lock can vouch for its own buffer, which the machine required to be empty, and for no one else's. So the cells it receives carry its completeness mark

only. The original points-to demands every processor's mark, which one acquisition cannot supply, so a general LOCK rule cannot promise it. The repaired calculus uses

$$e \rightarrowtail e' \;=_{df}\; \exists x : (e \rightsquigarrow_x e' \lhd \mathbf{bar}_x),$$

a flushed cell that is complete for whichever processor is credited with the write. This is the definition the dissertation considered first and rejected, on the grounds that $1 \rightsquigarrow_0 1 \lhd 2 \rightarrowtail 2$ would then have models in which the write to location 1 is still buffered. Those models are reachable, stable states of the machine. Because a model does not record who wrote a flushed cell, any marked processor may be credited, and two further axioms let any processor load from or store to such a cell. The original points-to remains expressible and keeps its meaning.

Two processors. The lock is represented only by the set of blocked processors. With a single processor there is no one to block: a held lock is indistinguishable from a free one, $\mathbf{lock}_i$ is equivalent to **emp**, and $\mathsf{lock}_i \,;\, \mathsf{lock}_i$ runs to completion. A rule that hands over the invariant at acquisition would then hand it over twice. On one processor the calculus derives a false specification of $\mathsf{lock}_i \,;\, \mathsf{lock}_i$ (`owned_soundness_without_two_processors_refuted`). The assumption is not about finiteness, and the second processor need not execute anything.

Roads not taken. Two other repairs were built during this work and abandoned, and they are the first two exits of Section 3.3. Keeping the invariant's assertions in a separate compartment of the judgment recovers the unrestricted FRAME-TM, but a cell can then never be returned to the invariant. Letting **emp** record cells that were given away permits the return, but $R \lhd \mathbf{emp}$ no longer entails R, and in that interpretation an ordinary later cell can silently erase an earlier, unrelated one.

5 The repaired model

The machine is unchanged. What changes is the predicate that says when a command is safe.

Resources. A thread's local state is a generalized memory ν with its *actual* completeness set. The original predicate read every local state at $\Gamma = \mathbb{P}$, which is why it could not distinguish a barrier the thread had earned from one it had not. A mark $i \in \Gamma$ now means what the dissertation said informally: nothing of processor i's is pending before this resource. A program step may add a mark only for its own processor, and only if it is fence_i, lock_i or unlock_i, whose guards require the whole of buffer i to be empty. Separately, the futures of a resource, under which safety is closed, include flushes of its own writes and growth of its marks; those concern temporal composition within the resource and do not certify that a buffer of the machine has drained.

Environments. The rest of the machine is described by a *pool* μ_J, holding the invariant's cells while the lock is free, and an external *frame* ν_F, standing for other threads and for any history that a frame rule has set aside. An environment for ν in mode m is a pool, a frame and a machine memory μ_0 with

$$\mu_0 \in \mu_J \mathbin{\widetilde{*}} (\nu_F \mathbin{\widehat{\#}} \nu)$$

such that either the lock is free in μ_0 and $(s, \mu_J, \mathbb{P}) \models J$, or the lock is held and μ_J is empty. In the held case the token may be in ν, in ν_F when another thread holds the lock, or split between them. The mode restricts the frame:

- in *overlap mode*, ov, any compatible frame is allowed, including one that mentions addresses of ν;
- in *disjoint mode*, dj, the frame and ν share no address.

Safety. $safe_{n+1}(c, s, \nu, J, Q)$ holds in mode m when, for every future $\nu_1 \leq \nu$:

1. if c is skip then $s, \nu_1 \models Q$; and
2. for every environment (μ_J, ν_F, μ_0) of ν_1, the configuration $c, (s, \mu_0)$ does not abort, and for every *program* step to $c', (s', \mu_1)$ there are ν' and μ'_J forming an environment of ν' with the *same* frame ν_F and memory μ_1, such that $safe_n(c', s', \nu', J, Q)$ and the step's bookkeeping conditions hold.

The bookkeeping conditions say that the pool changes only at lock and unlock; that unlock_i requires the whole token in ν_1; that the addresses of μ'_J and ν' lie within those of μ_J and ν_1; and that ν' gains no completeness mark beyond the one the step earns. A specification is valid in mode m, written $J \models^m \{P\}\ c\ \{Q\}$, when $s, \nu \models P$ implies safety at every n. Overlap-mode validity implies disjoint-mode validity. Appendix B gives these definitions in full.

Four features distinguish this from the original predicate.

- *The frame is fixed within a step and arbitrary between steps.* The original predicate let the frame take silent steps of its own choosing during a program step. That permits a local residual to claim a pending write that belonged to the frame, and the frame lemma fails (`overlap_frame_shape_counterexample`). Re-quantifying the frame at every step is what lets another thread, including one holding the lock, act in between.
- *Flushes are handled by closure.* A silent step is attributed to the frame or to the local resource with the other fixed (`pooled_resource_flush_attribution`), and safety is closed under futures of the local resource by its outermost quantifier.
- *The mark budget.* Without it, a proof could choose a successor that claims another processor's barrier.
- *The modes.* UNLOCK is valid only in disjoint mode: the released cells become the pool, and a frame holding a stale copy of one would collide with it. That is Proposition 1 seen from the semantics.

Three lemmas carry the proofs. With the frame fixed, the local residual's buffers are determined by the machine's: ordered overriding places the frame's writes to an address before the local ones, so there is one way to subtract the frame (`ordered_residual_unique`). Every machine flush belongs to exactly the frame or the local resource. And an invariant from the guarded fragment (Section 6) is buffer-free and holds of a memory under any nonempty set of completeness marks (`invariant_fragment_remark`), which is what allows the pool to be read at full completeness and handed over with the acquirer's mark alone.

6 The repaired calculus

The judgment is again $J \vdash \{P\}\ c\ \{Q\}$. The rules SKIP, ASSUME, ASSERT, FENCE, STORE, SEQ, CONS, DISJ, CHOICE, LOOP and FRAME-SP are those of Figure 1. Figure 2 shows the rest.

The temporal frame rule. FRAME-TM is the dissertation's rule with one more side condition: the command must not contain unlock, anywhere. The reason is the semantic one of Section 5. What the rules derive for a command that never releases is valid in overlap mode, where overlapping history is tolerated; the general UNLOCK rule is provided only in disjoint mode. The

$$\text{LOCK}\ \frac{}{J \vdash \{\mathbf{emp}\}\ \mathsf{lock}_i\ \{(\mathbf{lock}_i * J) \lhd \mathbf{bar}_i\}} \qquad \text{UNLOCK}\ \frac{A \lhd \mathbf{bar}_i \models J * B}{J \vdash \{\mathbf{lock}_i * A\}\ \mathsf{unlock}_i\ \{B\}}$$

$$\text{FRAME-TM}\ \frac{J \vdash \{P\}\ c\ \{Q\} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \emptyset \qquad c \text{ contains no } \mathsf{unlock}}{J \vdash \{R \lhd P\}\ c\ \{R \lhd Q\}}$$

$$\text{LOAD-ANY}\ \frac{x \notin \mathrm{fv}(J, e, e', P)}{J \vdash \{e \rightarrowtail e' \blacktriangleleft P\}\ x := [e]_i\ \{(e \rightarrowtail e' \blacktriangleleft P) \wedge x = e'\}}$$

$$\text{STORE-ANY}\ \frac{}{J \vdash \{e \rightarrowtail e'' \blacktriangleleft P\}\ [e] := e'_i\ \{(e \rightarrowtail e'' \blacktriangleleft P) \lhd e \rightsquigarrow_i e'\}} \qquad \text{ASSIGN}\ \frac{x \notin \mathrm{fv}(J)}{J \vdash \{P\,[e/x]\}\ x := e_i\ \{P\}}$$

$$\text{EX}\ \frac{J \vdash \{P\}\ c\ \{Q\} \qquad x \notin \mathrm{fv}(J, c, Q)}{J \vdash \{\exists x : P\}\ c\ \{Q\}}$$

Figure 2: Rules of the repaired calculus that differ from Figure 1. LOAD is the original axiom with $x \notin \mathrm{fv}(J, e, i, e', P)$, and CONC is the original rule with $\mathrm{mod}(c_k) \cap \mathrm{fv}(J) = \emptyset$ added.

mechanized judgment carries the mode as an index and has 21 rules, among them a temporal frame rule for overlap mode and a rule weakening overlap to disjoint (Appendix A). The judgment displayed here is its disjoint mode, and the displayed FRAME-TM is derived in it: a derivation for a command without unlock can be replayed in overlap mode, framed, and weakened (`owned_derive_temporal_frame_unlock_free`). The side condition is exact for the calculus: among the derivable specifications, those with an overlap-mode derivation are the ones whose command contains no unlock (`owned_overlap_derivable_iff_unlock_free`). It is also needed: FRAME-TM applied to a valid specification of a release can yield an invalid one (`owned_disjoint_temporal_frame_counterexample`).

Lock and unlock. LOCK delivers the invariant together with the acquirer's barrier. UNLOCK takes the whole of the thread's state A, notes that releasing drains the thread's buffer, and asks for a split into something satisfying J and a remainder B. Because A is arbitrary, nothing needs to be framed around a release: whatever the thread wants to keep goes into A and comes out in B.

Invariants. The soundness theorem requires J to come from the fragment

$$H \ ::= \ \mathbf{emp} \mid e \rightarrowtail e' \mid H * H \mid H \vee H \mid \exists x : H \mid b \wedge H.$$

A pure formula may appear only alongside a heap formula. A bare **true** is not buffer-free: it holds of a memory with pending writes, and an invariant containing it could swallow them at release. Some restriction is necessary: with the original $x \mapsto 7$ as invariant, LOCK derives a specification that is false, because one acquisition cannot supply every processor's mark (`owned_soundness_without_fragment_refuted`).

6.1 A proof system for assertions

The dissertation left entailment between assertions semantic. The judgment $P \vdash_a Q$ has 47 rules, none of which refers to satisfaction:

- the usual rules for $\wedge$, $\vee$, $\exists$, $\forall$, substitution and equality, and a rule admitting a valid implication between boolean expressions;
- for $*$ and $\lhd$: units, associativity, monotonicity, distribution over $\vee$ and $\exists$, movement of a pure conjunct in and out, and commutativity of $*$;
- the three adjunctions for the implications;
- for barriers, locks and writes, the surviving laws of Appendix D;
- two laws for cells: cells at distinct addresses compose spatially exactly when they compose temporally, and $e{\rightarrowtail}u \lhd e{\rightarrowtail}v \vdash_a e{\rightarrowtail}v$.

Strong temporal conjunction has no rules of its own. It is an abbreviation, and what is true of it follows from the rules for $\wedge$, $*$ and $\lhd$. The system is sound (`assertion_derivation_sound`) for all assertions and all processor domains. Exclusivity of cells, collapse of a drained history onto its last write, and hiding of a cell's value are derived in it, and the bound-variable renaming they need is itself proved. No completeness is claimed. Appendix E displays the rules that are particular to this logic.

What became of the derived rules. The dissertation also lists a dozen derived and alternative rules. Most are conveniences built from the primitive ones, and a few are proposals for stronger axioms. LOAD-P and STORE-P are the load and store axioms restated for the all-processor points-to $e \mapsto e'$. The rules whose names end in -G, such as FENCE-G and LOCK-G, take an arbitrary precondition P and append the command's effect to it; for example, FENCE-G is $J \vdash \{P\}\ \mathsf{fence}_i\ \{P \lhd \mathbf{bar}_i\}$. Those ending in -B are the backward forms, which start from an arbitrary postcondition. LOCKED and ATOMIC give a specification to a body run between acquiring and releasing the lock; ATOMIC, proposed as a stronger axiom, lets the body rely on the barrier that the release supplies when it restores the invariant.

In the repaired calculus FENCE-G is derived exactly as the dissertation displays it, and LOAD-P keeps its original form with the freshness of the loaded variable made explicit. LOCK-G, LOCK-B and the LOCKED wrapper are derived in a form that carries the transferred invariant. The unlock variants are derived for the empty invariant, the general case being UNLOCK itself. The "daring" rules for unlocked access to shared state remain unavailable. Appendix F states each rule in its repaired form.

7 Soundness and adequacy

Theorem 1 (Soundness, `owned_calculus_sound`)**.** *Let J be in the invariant fragment and let there be two distinct active processors. If $J \vdash \{P\}\ c\ \{Q\}$ is derivable then $J \models^{\mathsf{dj}} \{P\}\ c\ \{Q\}$, and if c contains no* unlock *then also $J \models^{\mathsf{ov}} \{P\}\ c\ \{Q\}$.*

The proof is by induction on the mode-indexed derivation, one semantic lemma per rule; the overlap-mode half first replays the derivation in overlap mode (`owned_derivation_unlock_free_overlap`). Both hypotheses are needed: dropping either admits a derivation of an invalid specification (Sections 4 and 6). No rule has a premise about safety or validity. CONS and UNLOCK have entailments as premises, which may be discharged semantically or in the proof system of Section 6.1.

Theorem 2 (Adequacy, `owned_derivation_adequacy`)**.** *Under the same hypotheses, let $J \vdash \{P\}\ c\ \{Q\}$ be derivable. Take a machine memory with the lock free that splits spatially into a*

pool satisfying J and a resource satisfying P. Then no finite execution of c from that memory aborts. If an execution terminates, its final memory splits into a pool and a resource satisfying Q, where either the lock is free and the pool satisfies J, or the resource holds the lock and the pool is empty.

Adequacy instantiates the external frame as empty, which satisfies both modes, so a client never discharges a mode condition.

Share. The original SHARE is unsound next to the transferring LOCK: a command that ends with the lock held has the invariant in its own hands, and SHARE would count it twice (`owned_original_share_lock_counterexample`). It is sound when every model of P and of Q is lock-free: then $J \models^m \{P\}\ c\ \{Q\}$ implies $\mathbf{emp} \models^m \{J * P\}\ c\ \{J * Q\}$, in either mode (`owned_share_valid`, `owned_share_overlap_valid`). The condition must be semantic; a postcondition that merely omits the token syntactically can still hide it under **true**.

For a specification from **emp** to **emp** there is a sharper statement (`owned_emp_derivation_restores`): if the external frame is lock-free to begin with, every completed execution ends with the lock free, and the final memory is exactly an invariant pool together with a flushed descendant of that frame.

These are partial-correctness results. They say nothing of termination, fairness or deadlock, and an execution stopped inside a critical section need not satisfy the invariant.

The standing hypotheses are used less than one might expect. The frame rules, the concurrency rule and the heap axioms need neither two processors nor, except for FRAME-TM, the fragment.

8 How proofs proceed

The dissertation gave few examples. This section gives five, in increasing size. All are derivations in the calculus of Section 6, checked by Rocq. Appendix G gives the programs and specifications of the larger ones in full.

8.1 A store, with and without a fence

For any invariant,

$$J \vdash \{x \rightarrowtail u\}\ [x] := v_i\,;\mathsf{fence}_i\ \{x \rightarrowtail v\}$$

is derivable (`owned_derive_store_fence_cell`): STORE-ANY gives $x \rightarrowtail u \lhd x \rightsquigarrow_i v$, FRAME-TM carries that history across fence_i, and a drained history collapses to its last write. This is the store rule of ordinary separation logic. The same specification without the fence is *invalid* (`owned_plain_store_cell_invalid`): the store may terminate with its write still buffered, and a cell assertion has no buffered writes. The logic is the difference between these two.

8.2 Reading a buffered write

After $[x] := v_i$ the thread holds $R \lhd x \rightsquigarrow_i v$, where R describes the older state of x. The load axiom wants the loaded write to be spatially separate from the rest, and an old cell cannot be separated spatially from a newer write to the same address. The temporal frame rule is what resolves this. The newest write alone satisfies the axiom's precondition, with **emp** for P; the older history is a temporal frame, provided r is free in none of J, R, x, i, v:

$$\frac{J \vdash \{x \rightsquigarrow_i v\}\ r := [x]_i\ \{x \rightsquigarrow_i v \wedge r = v\}}{J \vdash \{R \lhd x \rightsquigarrow_i v\}\ r := [x]_i\ \{R \lhd (x \rightsquigarrow_i v \wedge r = v)\}}\ \text{FRAME-TM}$$

(`owned_derive_load_after_history`). Store-to-load forwarding, the behaviour most characteristic of the machine, is thus an instance of the rule that most distinguishes the logic, and the reason it was worth keeping.

8.3 A shared container

One cell x is shared through a flag f. With $X = \exists v : x \rightarrowtail v$, the invariant is

$$J \;=\; f \rightarrowtail 0 \;\vee\; (f \rightarrowtail 1 * X).$$

A thread on processor 1 takes the item, writes to it privately, and returns it:

$$\begin{array}{l}\mathsf{lock}_1;\ r := [f]_1\,;\ \mathsf{assume}(r = 1)_1;\ [f] := 0_1;\ \mathsf{unlock}_1;\\ [x] := 9_1;\\ \mathsf{lock}_1;\ r := [f]_1\,;\ \mathsf{assume}(r = 0)_1;\ [f] := 1_1;\ \mathsf{unlock}_1\,.\end{array}$$

Meanwhile processor 2 has an unrelated write $R = y \rightsquigarrow_2 17$ buffered. The derived specification is $J \vdash \{R\}\ c\ \{R\}$ (`owned_container_derived`). Writing $H(a, v)$ for $(\exists u : a \rightarrowtail u) \lhd a \rightsquigarrow_1 v$, the proof passes through these assertions, each with R as a spatial frame:

After	The thread holds
first load and assume	$\mathbf{lock}_1 * (f \rightarrowtail 1 * X)$
$[f] := 0$	$\mathbf{lock}_1 * (H(f, 0) * X)$
first unlock	X
$[x] := 9$	$H(x, 9)$
second lock	$H(x, 9) \lhd ((\mathbf{lock}_1 * J) \lhd \mathbf{bar}_1)$
second load and assume	$\mathbf{lock}_1 * (f \rightarrowtail 0 * x \rightarrowtail 9)$
$[f] := 1$	$\mathbf{lock}_1 * (H(f, 1) * x \rightarrowtail 9)$
second unlock	$\mathbf{emp}$

Three things are visible in it.

The foreign write costs one rule application. R concerns an address the thread never touches, so a single FRAME-SP around the entire program carries it, across both releases.

The temporal frame rule carries the private write across the second acquisition. $H(x, 9)$ is overlapping history, possibly still buffered. FRAME-TM places it before the acquired state, and the barrier that LOCK delivers then drains it, leaving $x \rightarrowtail 9$.

Releases are justified by entailment. The two UNLOCK premises are $(H(f, 0) * X) \lhd \mathbf{bar}_1 \models J * X$ and $(H(f, 1) * x \rightarrowtail 9) \lhd \mathbf{bar}_1 \models J * \mathbf{emp}$. The first keeps the item and the second donates it.

The example is not vacuous. From a memory with the lock free, $f = 1$, $x = 7$ and processor 2's write pending, there is a complete 24-step execution in which that write is pending in every state (`owned_container_foreign_pending_throughout`), (`owned_container_trace`), and every execution from that memory is safe and, if it completes, restores the invariant (`owned_container_completed_restores`).

A second version replaces each $\mathsf{assume}()$ by a loop that releases the lock and retries when the flag is wrong (`owned_retrying_container_derived`). Its return loop has a branch for "the container is already full", and that branch is derived from **false**: the thread holds x, so the pool cannot.

The cost. The container's derivation applies CONS ten times and needs sixteen entailments that are not identities. Most are rearrangements, a few are the laws about barriers and histories used above, and the hardest is at the second acquisition. All sixteen are derived in the assertion proof system of Section 6.1, and none of its 47 rules had to be added to for the purpose. The same is true of the two examples that follow. The retrying version of the container has not been migrated and still discharges some entailments semantically.

8.4 The dissertation's example

The message-passing program of the dissertation's Section 4.1 is

$$\mathsf{lock}_1 ; [d] := 1_1 ; \mathsf{unlock}_1 \;\parallel\; \mathsf{lock}_2 ; x := [d]_2 ; \mathsf{unlock}_2 ; \mathsf{if}\ x = 1\ \mathsf{then}\ ([d] := 2_2 ; \mathsf{fence}_2)$$

and its claim is that from $d \mapsto 0$ every terminating execution ends with $d \mapsto x + 1$. The original proof tracked progress in two registers mentioned by the invariant. The repaired calculus forbids that, and uses ownership instead. Give the sender a token cell t and the receiver a token cell r, neither ever read or written, and take

$$J \;=\; d{\rightarrowtail}0 \;\vee\; (d{\rightarrowtail}1 * t{\rightarrowtail}0) \;\vee\; (t{\rightarrowtail}0 * r{\rightarrowtail}0).$$

A sender still holding t can only find the first disjunct when it acquires the lock, so its store is safe; it donates t with the updated cell. A receiver holding r cannot find the third. If it reads 1 it keeps d and donates r, and then writes to d with no lock. The derivation uses no auxiliary code and no semantic entailment.

The tokens are then removed from the conclusion. An execution from the original one-cell heap lifts to one with two extra untouched cells, and the machine never changes the set of allocated addresses, which rules out the one remaining disjunct in the $x = 0$ case. The result is a theorem about the original program and the original heap (`message_passing_all_runs`): no execution aborts, and every terminating one ends with $x \in \{0, 1\}$ and exactly the heap $d \mapsto x + 1$, which gives the dissertation's $d{\mapsto}x + 1$ (`message_passing_points_to`).

8.5 Named locks and a mailbox

The machine has one lock. Programs want many. A named lock is a flag cell l with an invariant I_l, represented in the global invariant by $(l{\rightarrowtail}0 * I_l) \vee l{\rightarrowtail}1$. Acquiring it is a loop: take the global lock, read the flag, set it if it was clear, release the global lock, and retry otherwise. For any finite family of such locks, with J the spatial product of their representations, the following are derived rules, given a scratch register that does not occur in J (`named_lock_acquire_family`, `named_lock_release_family`):

$$J \vdash \{\mathbf{emp}\}\ \mathit{acquire}(l)\ \{I_l\} \qquad\qquad \frac{I_l * I_l \vdash_a \mathbf{false}}{J \vdash \{I_l\}\ \mathit{release}(l)\ \{\mathbf{emp}\}}$$

The premise on release says that I_l cannot be held twice, which is how a thread holding I_l knows the flag is set. This is the lock rule of concurrent separation logic, recovered inside a logic with a single global lock. Between a named acquire and its release the global lock is free.

The client is a mailbox guarded by a named lock. A producer writes a message of K cells with ordinary stores and no fence, takes the named lock, marks the mailbox full, and releases. No fence is needed because the machine will not let it take the global lock, inside the named acquire, until its buffered writes have drained. A consumer retries until the mailbox is full, takes

the cells out, and reads them privately. The derived specification says that the consumer ends up owning cells that hold the values the producer wrote. It holds for every K, including zero (`mailbox_parallel_derived`), and by adequacy for every execution from an actual initial memory, with the flags, the message cells and one further address all distinct, in which a third processor's write to that address is pending (`mailbox_all_runs`). A checked 95-step execution of the two-cell instance leaves that write pending to the end.

9 The mechanization

The development is about 30,000 lines of Rocq in 196 files, using only the standard library. It declares no axioms and admits no proofs. An audit file prints the assumptions of 1,316 named results, and the build checks the compiled proof objects independently. The proofs need neither extensionality nor choice nor a finite set of processors. The development also keeps, as a record, the repairs that were tried and set aside.

The transcription of the original definitions is kept separate from every repair, so that every claim in Section 3 is a theorem about the dissertation's own definitions. Every Rocq name in this report refers to commit `1c66a28` of the development.

The work was done with AI assistance, in a loop that turned out to matter. Two models, Claude Fable and ChatGPT Astra, took part: one developed the proofs and the other reviewed them independently and proposed the design. Each rule or lemma was stated before it was proved, and where a bounded search was cheap it was run first. Twice the search or the proof attempt refuted a confident argument on paper: a frame lemma that let the frame evolve freely, and the soundness of FRAME-STM. Both now appear above as counterexamples. I do not think either would have been caught by reading.

10 Later work

This is a brief orientation, and it relies on the papers' own accounts of their results.

Before and alongside. Ridge [7] gave a rely-guarantee proof system for x86-TSO, mechanized in HOL4 and applied to Simpson's four-slot algorithm. Cohen and Schirmer [3] and Owens [6] took the other route: conditions on a program, an ownership discipline and triangular-race freedom respectively, under which its TSO executions are sequentially consistent, so that buffers need not be reasoned about at all. The dissertation, and the proposal with Berdine that preceded it [11], sat between these: buffers are visible in assertions, and separation is meant to keep the reasoning local.

iCAP-TSO. The closest successor is the logic of Sieczkowski, Svendsen, Birkedal and Pichon-Pharabod [8], which cites the dissertation. It extends a higher-order concurrent separation logic with store buffers and with connectives for them, including one stating that an assertion holds until a buffered update is flushed and another afterwards. That is a temporal connective in the dissertation's sense, attached to a single update. Their central idea is a *fiction of sequential consistency*: a library that synchronizes enough can be given a specification from which its clients reason in ordinary separation logic. They observe, fairly, that the dissertation's logic cannot exploit synchronization to simplify client proofs in that way. They also verify a spin lock whose release is a plain buffered store, which is exactly the unlocked access to shared state that Section 11 leaves open here. Jacobs [4] proposed "TSO spaces" for separation logic with a related aim.

Temporal reasoning elsewhere. Bornat, Alglave and Parkinson [2] built a rely-guarantee logic for weak memory around temporal modalities, "since" and "so far" among them. The idea that ordering between writes belongs in the assertion language, which the dissertation expressed as a separating conjunction, has thus reappeared in other forms.

Other memory models. Most later work targets the C11 model and weaker ones rather than TSO: relaxed separation logic [10] and GPS [9], and their reconstruction inside Iris [5], which made mechanized soundness proofs the norm for such logics. For x86 itself, Pierogi [1] is an Owicki–Gries logic over a view-based semantics that also covers persistent memory.

Where this report stands. Against that background, what the repaired logic offers is narrow and, as far as I know, still distinctive: sequential and spatial separating conjunctions over the whole history of a thread's writes, each with a frame rule, and a soundness proof checked end to end against an operational TSO machine. Section 3.3's conflict between temporal framing, units and ownership transfer does not depend on TSO and may be of use to anyone designing a logic with a sequential conjunction. What it lacks, unlocked access and a way for libraries to hide the memory model from their clients, is what the later logics were built to provide.

11 What remains open

Unlocked access to shared state. The dissertation opens with Dekker's algorithm, made correct on this machine by a fence after each store. Each thread reads a flag that the other writes, with no lock. The calculus cannot express that: a thread may touch a cell only if it owns it or holds the lock. The dissertation's basic "daring" rules were meant for this. They would let a thread load from or store to the invariant's cells without the lock, provided that the single command, specified with J in hand, gives J back. The dissertation doubted their soundness and left them as future work; they still are.

Realistic locks. The one lock is global and blocks every other processor. Programmer-level locks can be encoded as flag cells under the invariant, as the container does, but each release must itself take the global lock. A spinlock released by a bare store is another instance of unlocked access.

A strong temporal connective. The frame property holds for a single split that is both spatial and temporal (`owned_safe_common_frame`). Making that a connective of the assertion language, in place of the conjunction of two, would restore FRAME-STM. Whether its other laws would then hold is not known.

Assertions. The proof system for assertions is sound and was sufficient for every derivation attempted. Nothing is known about what it cannot prove, and nothing is automated.

The rest of the dissertation. Allocation, recursive predicates, permissions and the other topics of Chapter 5 are untouched, as is the sequential logic of Chapter 3.

References

[1] E. V. Bila, B. Dongol, O. Lahav, A. Raad and J. Wickerson. View-based Owicki–Gries reasoning for persistent x86-TSO. In *ESOP*, 2022.

[2] R. Bornat, J. Alglave and M. Parkinson. New lace and arsenic: adventures in weak memory with a program logic. arXiv:1512.01416, 2015.

[3] E. Cohen and B. Schirmer. From total store order to sequential consistency: a practical reduction theorem. In *ITP*, 2010.

[4] B. Jacobs. Verifying TSO programs. Report CW660, KU Leuven, 2014.

[5] J.-O. Kaiser, H.-H. Dang, D. Dreyer, O. Lahav and V. Vafeiadis. Strong logic for weak memory: reasoning about release-acquire consistency in Iris. In *ECOOP*, 2017.

[6] S. Owens. Reasoning about the implementation of concurrency abstractions on x86-TSO. In *ECOOP*, 2010.

[7] T. Ridge. A rely-guarantee proof system for x86-TSO. In *VSTTE*, 2010.

[8] F. Sieczkowski, K. Svendsen, L. Birkedal and J. Pichon-Pharabod. A separation logic for fictional sequential consistency. In *ESOP*, 2015.

[9] A. Turon, V. Vafeiadis and D. Dreyer. GPS: navigating weak memory with ghosts, protocols, and separation. In *OOPSLA*, 2014.

[10] V. Vafeiadis and C. Narayan. Relaxed separation logic: a program logic for C11 concurrency. In *OOPSLA*, 2013.

[11] I. Wehrman and J. Berdine. A proposal for weak-memory local reasoning. In *LOLA*, 2011.

[12] I. Wehrman. *Weak-Memory Local Reasoning*. PhD dissertation, The University of Texas at Austin, December 2012.

A The repaired calculus in full

Figure 3 gives the mechanized calculus, whose judgment $J \vdash^m \{P\}\ c\ \{Q\}$ carries a mode m, either ov or dj. The judgment of Section 6 is its disjoint mode.

SKIP
$$\frac{}{J \vdash^m \{P\}\ \mathsf{skip}_i\ \{P\}}$$

ASSUME
$$\frac{}{J \vdash^m \{\neg b \lor P\}\ \mathsf{assume}(b)_i\ \{P\}}$$

ASSERT
$$\frac{}{J \vdash^m \{b \land P\}\ \mathsf{assert}(b)_i\ \{P\}}$$

ASSIGN
$$\frac{x \notin \mathrm{fv}(J)}{J \vdash^m \{P\,[e/x]\}\ x := e_i\ \{P\}}$$

FENCE
$$\frac{}{J \vdash^m \{\mathbf{emp}\}\ \mathsf{fence}_i\ \{\mathbf{bar}_i\}}$$

LOCK
$$\frac{}{J \vdash^m \{\mathbf{emp}\}\ \mathsf{lock}_i\ \{(\mathbf{lock}_i * J) \lhd \mathbf{bar}_i\}}$$

UNLOCK
$$\frac{A \lhd \mathbf{bar}_i \models J * B}{J \vdash^{\mathsf{dj}} \{\mathbf{lock}_i * A\}\ \mathsf{unlock}_i\ \{B\}}$$

LOAD
$$\frac{x \notin \mathrm{fv}(J, e, i, e', P)}{J \vdash^m \{e \rightsquigarrow_i e' \blacktriangleleft P\}\ x := [e]_i\ \{(e \rightsquigarrow_i e' \blacktriangleleft P) \land x = e'\}}$$

STORE
$$\frac{}{J \vdash^m \{e \rightsquigarrow_i e'' \blacktriangleleft P\}\ [e] := e'_i\ \{(e \rightsquigarrow_i e'' \blacktriangleleft P) \lhd e \rightsquigarrow_i e'\}}$$

LOAD-ANY
$$\frac{x \notin \mathrm{fv}(J, e, e', P)}{J \vdash^m \{e \rightarrowtail e' \blacktriangleleft P\}\ x := [e]_i\ \{(e \rightarrowtail e' \blacktriangleleft P) \land x = e'\}}$$

STORE-ANY
$$\frac{}{J \vdash^m \{e \rightarrowtail e'' \blacktriangleleft P\}\ [e] := e'_i\ \{(e \rightarrowtail e'' \blacktriangleleft P) \lhd e \rightsquigarrow_i e'\}}$$

FRAME-SP
$$\frac{J \vdash^m \{P\}\ c\ \{Q\} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \emptyset}{J \vdash^m \{R * P\}\ c\ \{R * Q\}}$$

FRAME-TM
$$\frac{J \vdash^{\mathsf{ov}} \{P\}\ c\ \{Q\} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \emptyset}{J \vdash^{\mathsf{ov}} \{R \lhd P\}\ c\ \{R \lhd Q\}}$$

WEAKEN
$$\frac{J \vdash^{\mathsf{ov}} \{P\}\ c\ \{Q\}}{J \vdash^{\mathsf{dj}} \{P\}\ c\ \{Q\}}$$

CONS
$$\frac{P \models P' \qquad J \vdash^m \{P'\}\ c\ \{Q'\} \qquad Q' \models Q}{J \vdash^m \{P\}\ c\ \{Q\}}$$

SEQ
$$\frac{J \vdash^m \{P\}\ c\ \{R\} \qquad J \vdash^m \{R\}\ c'\ \{Q\}}{J \vdash^m \{P\}\ c\,;c'\ \{Q\}}$$

CHOICE
$$\frac{J \vdash^m \{P\}\ c\ \{Q\} \qquad J \vdash^m \{P\}\ c'\ \{Q\}}{J \vdash^m \{P\}\ c + c'\ \{Q\}}$$

LOOP
$$\frac{J \vdash^m \{P\}\ c\ \{P\}}{J \vdash^m \{P\}\ c^*\ \{P\}}$$

DISJ
$$\frac{J \vdash^m \{P\}\ c\ \{Q\} \qquad J \vdash^m \{P'\}\ c\ \{Q\}}{J \vdash^m \{P \lor P'\}\ c\ \{Q\}}$$

EX
$$\frac{J \vdash^m \{P\}\ c\ \{Q\} \qquad x \notin \mathrm{fv}(J, c, Q)}{J \vdash^m \{\exists x : P\}\ c\ \{Q\}}$$

CONC
$$\frac{J \vdash^m \{P_k\}\ c_k\ \{Q_k\} \qquad \mathrm{mod}(c_{1-k}) \cap \mathrm{fv}(J, P_k, c_k, Q_k) = \emptyset \quad (k = 0, 1)}{J \vdash^m \{P_0 * P_1\}\ c_0 \parallel c_1\ \{Q_0 * Q_1\}}$$

Figure 3: The 21 rules of the repaired calculus (`owned_derivation`). In LOCK and UNLOCK the processor is a constant.

B The safety predicate in full

This appendix spells out the definitions that Section 5 gives in prose (`owned_safe`, `owned_valid`).

Resources and futures. Every memory here blocks only active processors and has empty buffers at inactive ones. A *resource* $\nu = (\mu, \Gamma)$ is a generalized memory whose marks Γ are active processors, and whose heap is empty if Γ is. A memory *holds the lock for* i when i is active and the memory blocks exactly the other active processors. Its *futures*, written $\nu' \leq \nu$, are the resources reachable by any number of steps of two kinds:

- add marks: (μ, Γ) becomes (μ, Γ') with $\Gamma \subseteq \Gamma'$;
- flush: (μ, Γ) becomes (μ', Γ'), where μ' is μ after the oldest buffered write of some active processor i not blocked in μ moves to the heap, and $\Gamma \cup \{i\} \subseteq \Gamma'$.

Environments. An *environment* of ν at stack s, in mode m, consists of a pool μ_J, a frame resource ν_F, a machine memory μ_0 and a phase ϕ, which is either free or $\mathsf{held}(i)$, such that

1. $\mu_0 \in \mu_J \mathbin{\widetilde{*}} \mu$ for the memory μ of some resource in $\nu_F \mathbin{\widehat{\#}} \nu$;
2. in mode dj, the addresses of ν_F and of ν are disjoint;
3. if $\phi = \mathsf{free}$, no processor is blocked in μ_0 and $(s, \mu_J, \mathbb{P}) \models J$; and
4. if $\phi = \mathsf{held}(i)$, μ_0 holds the lock for i, and μ_J is empty.

Updates. A program step carries an action label α: fence by i, acquire by i, release by i, or ordinary. Stores, loads, assignments and the structural steps of choice, iteration and composition are ordinary. Let such a step take the local resource from $\nu = (\mu, \Gamma)$ to $\nu' = (\mu', \Gamma')$, the pool from μ_J to μ'_J and the phase from ϕ to ϕ'. The step is a permitted *update* when

1. *transfer:* if α is an acquire by i then $\phi = \mathsf{free}$, $\phi' = \mathsf{held}(i)$, μ'_J is empty and μ' holds the lock for i; if α is a release by i then $\phi = \mathsf{held}(i)$, $\phi' = \mathsf{free}$, μ holds the lock for i and μ' blocks no processor; otherwise $\phi' = \phi$, $\mu'_J = \mu_J$, and μ' blocks the same processors as μ;
2. *footprint:* every address of μ'_J or μ' is an address of μ_J or μ; and
3. *mark budget:* $\Gamma' \subseteq \Gamma \cup \{i\}$ if α is a fence, acquire or release by i, and $\Gamma' \subseteq \Gamma$ if α is ordinary.

Safety and validity. $\mathit{safe}_0(c, s, \nu, J, Q)$ always holds. $\mathit{safe}_{n+1}(c, s, \nu, J, Q)$ holds in mode m when, for every future $\nu_1 \leq \nu$:

1. if c is skip then $s, \nu_1 \models Q$; and
2. for every environment $(\mu_J, \nu_F, \mu_0, \phi)$ of ν_1 at s, the configuration $c, (s, \mu_0)$ does not abort, and for every program step from it to $c', (s', \mu_1)$ there are ν', μ'_J and ϕ' such that $(\mu'_J, \nu_F, \mu_1, \phi')$ is an environment of ν' at s', the step is a permitted update from (ν_1, μ_J, ϕ) to (ν', μ'_J, ϕ'), and $\mathit{safe}_n(c', s', \nu', J, Q)$.

$J \models^m \{P\}\ c\ \{Q\}$ holds when $s, \nu \models P$ implies $\mathit{safe}_n(c, s, \nu, J, Q)$ in mode m for every n. Flushes by the machine are not program steps. When J is in the invariant fragment the pool has no buffered writes, so each flush belongs either to the local resource or to the frame (`pooled_resource_flush_attribution`). A local flush is covered by the quantification over futures. A flush of the frame is covered by choosing the environment again, since the definition quantifies over all of them.

C The repaired definitions

Three definitions of the dissertation's model were replaced (Table 1). The first two admit fewer compositions than the originals, and the third adds a requirement to each implication.

Buffer overriding. Originally, $b \backslash\backslash b'$ is the set of interleavings of the buffers b and b' in which the most recent write to each address is the one that b' overriding b would give. That constrains only the last write to each address. In the repair, an interleaving is built from the oldest write onwards,

and a write to address ℓ may be taken from the later buffer b' only when no write to ℓ remains in b (`ordered_overrides`). All of b's writes to an address therefore precede all of b''s.

Temporal compatibility. Originally, μ_0 may precede μ_1 when three conditions hold. Their blocked sets are disjoint. For each active processor i *unblocked in both*, buffer i of μ_0 writes to no address in the heap of μ_1. For each pair of distinct active processors i and j *unblocked in both*, buffer i of μ_0 and buffer j of μ_1 write to disjoint sets of addresses. The repair drops "unblocked in both" from the second and third conditions (`stable_compatible`).

Implications. Originally, ν satisfies $P \mathrel{-\!\!*} Q$ when every composition of ν with a resource satisfying P satisfies Q, and similarly for the two temporal implications. In the repair the same is required of every future of ν, so that an implication which holds now cannot be falsified by a flush. This applies at every implication in an assertion, nested ones included. It does not follow that a repaired assertion has fewer models than the original one, because the antecedent of an implication is reinterpreted too.

D The dissertation's assertion laws

The dissertation states its laws for assertions in two figures, one of equivalences and one of entailments (Figures 4.5 and 4.6). Table 3 classifies every one of them in the final interpretation. A proved law holds for all assertions, implications included, and all processor domains. Each refuted law is also refuted in the dissertation's original interpretation.

Law ($\bullet$ ranges over $*$, $\lhd$, $\blacktriangleleft$)	Status	Note
$P \bullet \mathbf{emp} \equiv P \equiv \mathbf{emp} \bullet P$	proved	all three
$(P \bullet Q) \bullet R \equiv P \bullet (Q \bullet R)$	proved for $*$, $\lhd$	refuted for $\blacktriangleleft$, both directions
$P * Q \equiv Q * P$	proved	
$P \bullet \mathbf{lock}_e \equiv \mathbf{lock}_e \bullet P$	proved	indeed $P \lhd \mathbf{lock}_e \equiv P * \mathbf{lock}_e$
$\mathbf{lock}_e \bullet \mathbf{lock}_f \equiv \mathbf{false}$	qualified	needs a processor blocked by both
$\mathbf{bar}_e \bullet \mathbf{bar}_e \equiv \mathbf{bar}_e$	proved	
$(P \bullet Q) \lhd \mathbf{bar}_e \equiv (P \lhd \mathbf{bar}_e) \bullet (Q \lhd \mathbf{bar}_e)$	proved	all three
$P \blacktriangleleft \mathbf{bar}_e \equiv P \lhd \mathbf{bar}_e$	proved	
$\mathbf{lock}_e \lhd \mathbf{bar}_e \equiv \mathbf{lock}_e * \mathbf{bar}_e$	proved	
$e \rightsquigarrow_{e'} e'' * e \rightsquigarrow_{f'} f'' \equiv \mathbf{false}$	proved	
$e \rightsquigarrow_i e' \lhd e \rightsquigarrow_i f \lhd \mathbf{bar}_i \equiv e \rightsquigarrow_i f \lhd \mathbf{bar}_i$	proved	
$\mathbf{bar}_e \models \mathbf{emp}$; monotonicity of $\bullet$	proved	
$e \mapsto f \models e \rightsquigarrow_{e'} f$	qualified	e' must denote an active processor
$P \blacktriangleleft Q \models P * Q$ and $P \blacktriangleleft Q \models P \lhd Q$	proved	
Small exchange, $\blacktriangleleft$ outermost	refuted	under either reading of its premise
Small exchange between $*$ and $\lhd$	refuted or vacuous	refuted for the premise as displayed; read as an inclusion of connectives, the premise forces an empty processor set
Full exchange for $*$ and $\blacktriangleleft$	refuted	

Table 3: The original assertion laws, classified.

The pattern is simple to state. The laws that rebracket or exchange through $\blacktriangleleft$ fail, for the reason FRAME-STM fails: the two conjuncts of $P \blacktriangleleft Q$ choose their splits independently. No exchange law between different conjunctions is usable. The remaining laws survive, two of them

with a side condition. The unit laws needed the most care, since composing with an empty resource can add completeness marks, and the proof must show that no assertion, however many implications it contains, can tell.

E A selection of the assertion rules

Figure 4 shows the rules of $P \vdash_a Q$ that are particular to this logic. The figure omits the usual structural and logical rules, substitution and equality, a rule admitting valid implications between boolean expressions and a case split on a boolean, the distribution of $*$ and $\lhd$ over $\vee$ and $\exists$, and the mirror image of the rule for pure conjuncts. In the figure $\bullet$ stands for either $*$ or $\lhd$, the same one throughout a rule, and $P \dashv\vdash_a Q$ abbreviates the two rules $P \vdash_a Q$ and $Q \vdash_a P$.

$$\mathbf{emp} \bullet P \dashv\vdash_a P \dashv\vdash_a P \bullet \mathbf{emp} \qquad (P \bullet Q) \bullet R \dashv\vdash_a P \bullet (Q \bullet R) \qquad P * Q \vdash_a Q * P \qquad \frac{P \vdash_a P' \qquad Q \vdash_a Q'}{P \bullet Q \vdash_a P' \bullet Q'}$$

$$(b \wedge P) \bullet Q \dashv\vdash_a b \wedge (P \bullet Q) \qquad \frac{P * Q \vdash_a R}{Q \vdash_a P \mathrel{-\!\!*} R} \qquad \frac{P \lhd Q \vdash_a R}{Q \vdash_a P \mathrel{-\!\!\lhd} R} \qquad \frac{Q \lhd P \vdash_a R}{Q \vdash_a R \mathrel{\lhd\!\!-} P}$$

$$\mathbf{bar}_e \vdash_a \mathbf{emp} \qquad \mathbf{bar}_e \bullet \mathbf{bar}_e \dashv\vdash_a \mathbf{bar}_e \qquad P \lhd \mathbf{bar}_e \vdash_a P * \mathbf{bar}_e$$

$$(P \bullet Q) \lhd \mathbf{bar}_e \dashv\vdash_a (P \lhd \mathbf{bar}_e) \bullet (Q \lhd \mathbf{bar}_e)$$

$$P \lhd \mathbf{lock}_e \dashv\vdash_a P * \mathbf{lock}_e \qquad \mathbf{lock}_e \lhd P \dashv\vdash_a \mathbf{lock}_e * P \qquad \frac{k \text{ active} \qquad k \neq i \qquad k \neq j}{\mathbf{lock}_i * \mathbf{lock}_j \vdash_a \mathbf{false}}$$

$$e \rightsquigarrow_f e' * e \rightsquigarrow_{f'} e'' \vdash_a \mathbf{false} \qquad (e \rightsquigarrow_f e' \lhd e \rightsquigarrow_f e'') \lhd \mathbf{bar}_f \dashv\vdash_a e \rightsquigarrow_f e'' \lhd \mathbf{bar}_f$$

$$\frac{f \text{ denotes an active processor in every stack}}{e \mapsto e' \vdash_a e \rightsquigarrow_f e'}$$

$$e \neq f \wedge (e \rightarrowtail e' \lhd f \rightarrowtail f') \dashv\vdash_a e \neq f \wedge (e \rightarrowtail e' * f \rightarrowtail f') \qquad e \rightarrowtail e' \lhd e \rightarrowtail e'' \vdash_a e \rightarrowtail e''$$

Figure 4: Assertion rules particular to this logic, from the 47 of `assertion_derivation`. Each implication rule is invertible; only the introduction direction is shown.

A few of these do most of the work in the derivations of Section 8. The barrier rules say that a barrier is an empty resource that can be duplicated, and that draining a composite drains each part. The lock rules say that a lock assertion carries no history, so its temporal and spatial compositions agree. The collapse rule for writes says that once a processor's writes in a history have drained, only its last write to an address is visible. The two cell rules are what make cells behave like ordinary heap cells: cells at different addresses can be reordered freely, and a later cell at the same address overwrites an earlier one.

F The dissertation's derived rules

Most of the dissertation's derived rules are conveniences built from the primitive ones. Figure 5 states them as they are derived in the repaired calculus, and Table 4 records how each differs from its original display. LOAD-P and STORE-P are the load and store axioms for the all-processor

points-to $e \mapsto e'$, so that a thread can use a cell without naming the processor that wrote it. The -G rules run forwards from an arbitrary precondition P and append the command's effect to it. The -B rules run backwards from an arbitrary postcondition Q, and their precondition says that Q will hold once the command's effect is appended. LOCKED gives a specification to a body run between acquiring and releasing the lock.

$$\textsc{load-p}\ \frac{x \notin \mathrm{fv}(J, e, e', P)}{J \vdash \{e \mapsto e' \blacktriangleleft P\}\ x := [e]_i\ \{(e \mapsto e' \blacktriangleleft P) \wedge x = e'\}}$$

$$\textsc{store-p}\ \frac{}{J \vdash \{e \mapsto e'' \blacktriangleleft P\}\ [e] := e'_i\ \{(e \mapsto e'' \blacktriangleleft P) \lhd e \rightsquigarrow_i e'\}} \qquad \textsc{fence-g}\ \frac{}{J \vdash \{P\}\ \mathsf{fence}_i\ \{P \lhd \mathbf{bar}_i\}}$$

$$\textsc{fence-b}\ \frac{}{J \vdash \{Q \lhd\!\!- \mathbf{bar}_i\}\ \mathsf{fence}_i\ \{Q\}} \qquad \textsc{lock-g}\ \frac{}{J \vdash \{P\}\ \mathsf{lock}_i\ \{P \lhd ((\mathbf{lock}_i * J) \lhd \mathbf{bar}_i)\}}$$

$$\textsc{lock-b}\ \frac{}{J \vdash \{Q \lhd\!\!- ((\mathbf{lock}_i * J) \lhd \mathbf{bar}_i)\}\ \mathsf{lock}_i\ \{Q\}} \qquad \textsc{unlock-g}\ \frac{}{\mathbf{emp} \vdash \{P * \mathbf{lock}_i\}\ \mathsf{unlock}_i\ \{P \lhd \mathbf{bar}_i\}}$$

$$\textsc{unlock-b}\ \frac{}{\mathbf{emp} \vdash \{(Q \lhd\!\!- \mathbf{bar}_i) * \mathbf{lock}_i\}\ \mathsf{unlock}_i\ \{Q\}}$$

$$\textsc{locked}\ \frac{J \vdash \{P \lhd ((\mathbf{lock}_i * J) \lhd \mathbf{bar}_i)\}\ c\ \{\mathbf{lock}_i * A\} \qquad A \lhd \mathbf{bar}_i \models J * Q}{J \vdash \{P\}\ \mathsf{lock}_i\,;c\,;\mathsf{unlock}_i\ \{Q\}}$$

Figure 5: The dissertation's derived rules in the form derived in the repaired calculus. In the lock rules the processor is a constant.

Original rule	In the repaired calculus
LOAD-P	Derived with the original triple and the original points-to; the freshness of x is made explicit.
STORE-P	The display has a load command and a strong temporal suffix, which no memory satisfies; with a store command and a temporal suffix, as in STORE, it is derived.
Strengthened LOCK, UNLOCK	Subsumed: LOCK already delivers $\mathbf{bar}_i$, and UNLOCK's premise already assumes it.
FENCE-G	Derived as displayed.
LOCK-G, LOCK-B	Derived with the transferred invariant made explicit.
UNLOCK-G, UNLOCK-B	Derived for the empty invariant; in general the thread cannot promise to keep all of P while donating part of it.
FENCE-B	The display has lock where fence is meant; with fence it is derived.
LOCKED, ATOMIC	Derived as a wrapper whose body is proved under the same J. Importing a body proved under **emp** is not recovered.
Daring load and store	Not available (Section 11).

Table 4: Status of the rules in the dissertation's "Derived and Alternative Axioms and Inference Rules".

G The programs of Section 8

This appendix gives the programs and specifications that Section 8 describes in prose. Conditionals and loops are the dissertation's abbreviations for choice, iteration and assume().

The retrying container. With $J = f{\rightarrowtail}0 \vee (f{\rightarrowtail}1 * X)$ as in Section 8, taking the item is

$$\begin{aligned} \mathit{take} \;=\; & g := 0_1;\ \mathsf{while}\ \ g = 0\ \ \mathsf{do} \\ & (\mathsf{lock}_1;\ r := [f]_1\,;\ \mathsf{if}\ \ r = 1\ \ \mathsf{then}\ \ ([f] := 0_1;\ \mathsf{unlock}_1;\ g := 1_1)\ \ \mathsf{else}\ (\mathsf{unlock}_1;\ g := 0_1)) \end{aligned}$$

and *put* is the same with the test $r = 0$ and the store $[f] := 1_1$. The program is *take*; $[x] := 9_1$; *put*, and the derived specification is $J \vdash \{R\}\ c\ \{R\}$ with $R = y \rightsquigarrow_2 17$ (`owned_retrying_container_derived`).

Message passing. With J as in Section 8.4, the two threads and their composition have the specifications

$$\begin{aligned} & J \vdash \{t{\rightarrowtail}0\}\ \ \mathsf{lock}_1\,;[d] := 1_1\,;\mathsf{unlock}_1\ \ \{\mathbf{emp}\} \\ & J \vdash \{r{\rightarrowtail}0\}\ \ \mathit{receiver}\ \ \{(r{\rightarrowtail}0 \wedge x = 0) \vee (d{\rightarrowtail}2 \wedge x = 1)\} \\ & J \vdash \{t{\rightarrowtail}0 * r{\rightarrowtail}0\}\ \ \mathit{sender} \parallel \mathit{receiver}\ \ \{(r{\rightarrowtail}0 \wedge x = 0) \vee (d{\rightarrowtail}2 \wedge x = 1)\} \end{aligned}$$

(`mp_sender_derived`, `mp_receiver_derived`, `mp_parallel_derived`). The sender gives up its token. The receiver ends either still holding its token, having read 0, or owning the data cell with the value 2, having read 1.

Named locks. For a flag cell l, a processor i and a scratch register x:

$$\begin{aligned} \mathit{acquire}_i(l,x) \;=\; & x := 1_i;\ \mathsf{while}\ \ x = 1\ \ \mathsf{do} \\ & (\mathsf{lock}_i;\ x := [l]_i\,;\ \mathsf{if}\ \ x = 0\ \ \mathsf{then}\ \ ([l] := 1_i;\ \mathsf{unlock}_i)\ \ \mathsf{else}\ \mathsf{unlock}_i) \\ \mathit{release}_i(l) \;=\; & \mathsf{lock}_i;\ [l] := 0_i;\ \mathsf{unlock}_i \end{aligned}$$

Under the invariant below the flag is always 0 or 1, so the loop exits with $x = 0$, the value read from a flag that was clear and is now set. For locks $l_1, \ldots, l_n$ with invariants $I_1, \ldots, I_n$, the global invariant is

$$J \;=\; \big((l_1{\rightarrowtail}0 * I_1) \vee l_1{\rightarrowtail}1\big) * \cdots * \big((l_n{\rightarrowtail}0 * I_n) \vee l_n{\rightarrowtail}1\big),$$

and the two derived rules of Section 8 hold for each l_k, given $x \notin \mathrm{fv}(J)$.

The mailbox. There is one named lock l, a flag cell m that says whether the mailbox is full, and message cells $a_1, \ldots, a_K$ that are to receive the values $v_1, \ldots, v_K$. Write $A(\vec{w})$ for $a_1{\rightarrowtail}w_1 * \cdots * a_K{\rightarrowtail}w_K$, which is **emp** when $K = 0$. The lock's invariant is

$$I_l \;=\; m{\rightarrowtail}0 \;\vee\; (m{\rightarrowtail}1 * A(\vec{v})),$$

and $J = (l{\rightarrowtail}0 * I_l) \vee l{\rightarrowtail}1$. The producer runs on processor 1 and the consumer on processor 2; the processor annotations on their stores, loads and assignments are omitted here:

$$\begin{aligned} \mathit{producer} \;=\; & [a_1] := v_1;\ \ldots;\ [a_K] := v_K;\ \mathit{acquire}_1(l,z);\ [m] := 1;\ \mathit{release}_1(l) \\ \mathit{consumer} \;=\; & y := 1;\ \mathsf{while}\ \ y = 1\ \ \mathsf{do} \\ & \quad (\mathit{acquire}_2(l,y);\ y := [m]\,; \\ & \quad\ \ \mathsf{if}\ \ y = 1\ \ \mathsf{then}\ \ ([m] := 0;\ \mathit{release}_2(l);\ y := 0)\ \ \mathsf{else}\ (\mathit{release}_2(l);\ y := 1)); \\ & w := [a_1]\,;\ \ldots;\ w := [a_K] \end{aligned}$$

The derived specifications are

$$J \vdash \left\{A(\vec{0})\right\}\ \mathit{producer}\ \{\mathbf{emp}\}$$
$$J \vdash \{\mathbf{emp}\}\ \mathit{consumer}\ \{A(\vec{v})\}$$
$$J \vdash \left\{A(\vec{0})\right\}\ \mathit{producer} \parallel \mathit{consumer}\ \{A(\vec{v})\}$$

(`mailbox_producer_derived`, `mailbox_consumer_derived`, `mailbox_parallel_derived`). The producer starts with the message cells, all zero, and ends with nothing: it has put them in the mailbox. The consumer starts with nothing and ends owning the message cells with the producer's values.